



(۱۵ نمره)

۱. سؤال ۱

دو معماری A و B را در نظر بگیرید: (same = Padding است)

- A
Input: $32 \times 32 \times 3$
Conv: $Kernel = 3 \times 3, Filters = 16$
MaxPool: $Kernel = 2 \times 2$
Conv: $Kernel = 3 \times 3, Filters = 32$
MaxPool: $Kernel = 2 \times 2$
Flatten:
FC: $2048 \rightarrow 128$
FC: $128 \rightarrow 10$
- B
Input: $32 \times 32 \times 3$
Conv: $kernel = 3 \times 3, Filters = 16$
MaxPool: $kernel = 2 \times 2$
Conv: $kernel = 3 \times 3, Filters = 32$
MaxPool: $kernel = 2 \times 2$
Conv: $kernel = 1 \times 1, Filters = 128$
Global Average Pooling
Conv: $kernel = 1 \times 1, Filters = 10$

- (آ) تفاوت اصلی دو معماری در چیست؟ نقش لایه Flatten در معماری A را بیان کنید و بگویید چرا در معماری B به آن نیازی نداریم؟
- (ب) تعداد پارامترهای هر لایه در این دو معماری را بدست آورده و تعداد کل پارامترها در این دو معماری را مقایسه کنید.
- (ج) میزان محاسبات مورد نیاز برای هر دو معماری را بدست آورده و با یکدیگر مقایسه کنید.
- (د) فرض کنید اندازه ورودی از 32×32 به 64×64 تغییر می‌یابد. کدام معماری بدون تغییر ساختار قابل استفاده است همچنان؟ دلیل این موضوع را بیان کنید.
- (ه) کدامیک از دو معماری بالا اطلاعات مکانی را بهتر حفظ می‌کند؟ کدامیک برای Segmentation مناسب‌تر است؟

(۱۵ نمره)

۲. سؤال ۲

سه مدل SAM، SegNet و U-Net را در نظر بگیرید.

- (آ) برای هر کدام از سه مدل بالا ساز و کار و ساختار معماری‌ها را به صورت کامل شرح دهید.
- (ب) اگر هدف جداسازی رگ‌های خونی نازک باشد بین U-Net و SegNet کدام معماری عملکرد بهتری دارد؟ دلیل خود را بنویسید. نقش و اهمیت Skip-Connection در این مسئله را نیز توضیح دهید.
- (ج) کدام معماری حافظه بیشتری در زمان آموزش مصرف می‌کند؟ کدام معماری برای GPU کوچک مناسب‌تر است؟ علت این تفاوت در چیست؟
- (د) برای هر معماری توضیح دهید که آیا به داده برچسب خورده نیاز است؟ حجم داده مورد نیاز برای آموزش موثر چقدر است؟ آیا می‌توان از مدل پیش‌آموزش‌دیده استفاده کرد؟
- (ه) برای هر سناریو معماری مناسب را انتخاب کنید.
- ناحیه‌بندی تومور مغزی با تنها ۵۰۰ تصویر برچسب خورده.
 - ناحیه‌بندی اشیاء شناخته‌شده در تصاویر اینترنتی بدون آموزش مجدد
 - یک سامانه بلادرنگ روی یک دستگاه Embedded با حافظه محدود
 - بخش‌بندی دقیق مرز سلول‌ها در تصاویر میکروسکوپی

(۲۵ نمره)

۳. سؤال ۳

ارزیابی سرتاسری: سرکوب غیربیشینه (NMS)، معیارها، و دقت متوسط (AP)

شرح مسئله:

شما در حال ارزیابی یک مدل آموزش‌دیده تشخیص اشیاء هستید. در یک تصویر تست مشخص، دقیقاً ۴ شیء برچسب‌خورده (Ground-Truth) وجود دارد (GT_1, GT_2, GT_3, GT_4) .

مدل ۹ کادر محصورکننده (Bounding Box) پیشنهادی خام خروجی می‌دهد $(P_1$ تا $P_9)$. پیش‌بینی‌ها بر اساس امتیاز اطمینان (Confidence Score) به ترتیب نزولی مرتب شده‌اند:

- P_1 (اطمینان: 0.99)
- P_2 (اطمینان: 0.96)
- P_3 (اطمینان: 0.91)
- P_4 (اطمینان: 0.89)
- P_5 (اطمینان: 0.84)
- P_6 (اطمینان: 0.72)
- P_7 (اطمینان: 0.56)
- P_8 (اطمینان: 0.38)
- P_9 (اطمینان: 0.25)

بخش الف: محاسبه سرکوب غیربیشینه (NMS)

قبل از ارزیابی مدل در برابر مقادیر واقعی (Ground Truth)، باید از NMS برای فیلتر کردن پیش‌بینی‌های اضافی استفاده کنید، زیرا ناحیه‌هایی با اشتراک به اجتماع (IoU) بزرگ ترکیب یا حذف می‌شوند. آستانه همپوشانی NMS برابر با $\text{IoU} \geq 0.5$ تنظیم شده است.

با توجه به داده‌های همپوشانی داخلی زیر بین پیش‌بینی‌های خود مدل، دقیقاً مشخص کنید کدام پیش‌بینی‌ها از فیلتر NMS عبور کرده (باقی می‌مانند) و کدام یک سرکوب (حذف) می‌شوند. منطق و مراحل کار خود را به صورت گام‌به‌گام نشان دهید.

- P_1 و P_2 دارای همپوشانی با IoU داخلی 0.85 هستند.
- P_1 و P_3 دارای همپوشانی با IoU داخلی 0.65 هستند.
- P_2 و P_3 دارای همپوشانی با IoU داخلی 0.45 هستند.
- P_4 و P_6 دارای همپوشانی با IoU داخلی 0.55 هستند.
- P_5 و P_9 دارای همپوشانی با IoU داخلی 0.30 هستند.
- P_7 و P_8 دارای همپوشانی با IoU داخلی 0.75 هستند.
- سایر ترکیبات دو به دوی پیش‌بینی‌ها از نظر مکانی مجزا هستند و IoU داخلی آن‌ها 0.0 است.

بخش ب: دسته‌بندی تشخیص‌ها (FN, FP, TP)

اکنون مدل را در برابر مقادیر واقعی ارزیابی کنید. آستانه ارزیابی برای معتبر شمردن یک تشخیص برابر با $\text{IoU} \geq 0.5$ تنظیم شده است.

در زیر همپوشانی‌های مکانی بین هر ۹ پیش‌بینی خام و کادرهای برچسب‌خورده واقعی آورده شده است.

- پیش‌بینی P_1 دارای IoU معادل 0.90 با GT_1 است.
- پیش‌بینی P_2 دارای IoU معادل 0.88 با GT_1 است.
- پیش‌بینی P_3 دارای IoU معادل 0.70 با GT_1 است.
- پیش‌بینی P_4 دارای IoU معادل 0.85 با GT_2 است.
- پیش‌بینی P_5 دارای IoU معادل 0.45 با GT_3 است.
- پیش‌بینی P_6 دارای IoU معادل 0.60 با GT_2 است.
- پیش‌بینی P_7 دارای IoU معادل 0.80 با GT_4 است.
- پیش‌بینی P_8 دارای IoU معادل 0.75 با GT_4 است.
- پیش‌بینی P_9 دارای IoU معادل 0.00 با تمامی مقادیر واقعی است (پس‌زمینه).

وظیفه: با استفاده از تنها پیش‌بینی‌هایی که از بخش الف باقی مانده‌اند، هر پیش‌بینی معتبر را به عنوان یک تشخیص صحیح (TP) یا تشخیص اشتباه (FP) دسته‌بندی کنید. سپس، تعداد کل نهایی برای TP، FP، و تشخیص‌های از دست‌رفته (False Negatives یا FN) را تعیین کنید. توضیح دهید که چگونه اعمال نتایج NMS از بخش الف، مانع از شمارش بیش از حد FP‌ها شد.

بخش پ: محاسبه معیارهای استاندارد

با استفاده از تعداد نهایی TP، FP، و FN محاسبه شده از پیش‌بینی‌های باقی‌مانده در بخش ب، معیارهای کلی زیر را برای این تصویر محاسبه کنید. محاسبات دقیق ریاضی خود را بر اساس فرمول‌های استاندارد نشان دهید:

$$\text{دقت (Precision)}: \text{Precision} = \frac{TP}{TP+FP}$$

$$\text{ب) فراخوانی (Recall)}: \text{Recall} = \frac{TP}{TP+FN}$$

$$\text{ج) امتیاز اف-یک (Score F1)}: F_1 = 2 \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

بخش ت: محاسبه دقت متوسط (AP)

دقت متوسط (Average Precision) مستلزم ردیابی نحوه تغییر Precision و Recall در هنگام پذیرش متوالی پیش‌بینی‌های باقی‌مانده بر اساس رتبه اطمینان آن‌هاست.

آ) جدولی ایجاد کنید که Precision و Recall تجمعی را در هر مرحله محاسبه کند و پیش‌بینی‌های باقی‌مانده را به ترتیب امتیاز اطمینان (از بیشترین به کمترین) پردازش کند.

ب) با استفاده از مقادیر جدول خود، مقدار دقیق دقت متوسط (AP) را با یافتن مساحت زیر منحنی پله‌ای خام Precision-Recall محاسبه کنید. مساحت زیر منحنی (AP) از نظر ریاضی با انتگرال $AP = \int_0^1 p(r) dr$ تعریف می‌شود. (فرض کنید منحنی برای هر مقدار Recall بیشتر از حداکثر Recall به دست آمده، به Precision معادل صفر افت می‌کند).

بخش ث: تحلیل تئوری

بر اساس محاسبات انجام شده در بالا و دانش خود از خط لوله‌های (Pipelines) تشخیص اشیاء، به موارد زیر پاسخ دهید:

آ) نقش دوگانه IoU: شما از معیار IoU یک‌بار در بخش الف و مجدداً در بخش ب استفاده کردید. تفاوت عملیاتی بین محاسبه IoU انجام شده در طول NMS در مقابل مرحله ارزیابی را به وضوح توضیح دهید. دقیقاً چه چیزی در هر یک از این مراحل با هم مقایسه می‌شود؟

ب) درونیابی منحنی (Curve Interpolation): در سیستم‌های ارزیابی عملی، معمولاً از مساحت زیر یک منحنی «تعدیل‌شده (Adjusted)» یا درونیابی شده به جای منحنی پله‌ای خام محاسبه شده در بخش ت استفاده می‌شود. توضیح دهید منحنی تعدیل‌شده Precision-Recall چیست و چرا این درونیابی برای یک ارزیابی منصفانه از مدل مفید است.

ج) تأثیر حذف NMS: اگر بخش الف را به طور کامل نادیده بگیرید و هر ۹ پیش‌بینی خام را مستقیماً در برابر مقادیر واقعی ارزیابی کنید، تأثیر تئوری آن را بر تعداد تشخیص‌های اشتباه (FP)، معیار دقت (Precision) و AP کلی مدل استنتاج کنید.

(۲۵ نمره)

۴. سوال ۴

تحلیل معماری‌های تشخیص شیء و پیچیدگی محاسباتی

در این مسئله، ابتدا ابعاد و پیچیدگی محاسباتی شبکه‌ی VGG16 را به عنوان یک استخراج‌کننده‌ی ویژگی (Backbone) بررسی می‌کنیم و سپس روند تکامل شبکه‌های تشخیص شیء مبتنی بر ناحیه را

از منظر رفع گلوگاه‌های محاسباتی و معماری ارزیابی می‌نماییم. با توجه به این رویکرد، به بخش‌های زیر به طور دقیق پاسخ دهید:

(الف) جدول انتهایی جزئیات ساختاری شبکه VGG16 را نشان می‌دهد. با توجه به اطلاعات مندرج در این جدول، به پرسش‌های زیر پاسخ دهید:

۱. در صورتی که لایه‌های کاملاً متصل این شبکه را به لایه‌های کانولوشنی (padding) تبدیل کنیم و در ورودی شبکه یک تصویر 512×512 قرار دهیم، خروجی شبکه چه ابعادی خواهد داشت؟
۲. لایه ابتدایی این شبکه شامل 64 فیلتر 3×3 است. اگر تصویر ورودی شبکه 512×512 باشد، این لایه چه تعداد ضرب و جمع نیاز دارد (به طور دقیق محاسبه کنید و مراحل محاسبه را بنویسید)؟
۳. اگر بجای پیاده‌سازی کانولوشنی، شبکه VGG16 را بر روی همان مکان‌هایی که در قسمت قبل اعمال شده است، به صورت عادی اعمال کنیم، تعداد ضرب و جمع لازم چه مقدار خواهد بود؟ (در این قسمت تعداد ضرب و جمع در حالت عادی با ورودی 224×224 را محاسبه کنید و این عدد را در تعداد دفعاتی که لازم است شبکه اجرا شود ضرب کنید). نتیجه این قسمت را با قسمت قبل مقایسه کنید. محاسبات تنها برای لایه اول صورت گیرد.

(ب) گلوگاه معماری (R-CNN در مقابل Fast R-CNN)

معماری VGG16 که در بخش‌های قبلی تحلیل شد، اغلب به عنوان ستون فقرات استخراج ویژگی (Backbone Feature Extractor) در سیستم‌های تشخیص شیء دو مرحله‌ای استفاده می‌شود. در چارچوب اولیه R-CNN، یک روش پیشنهاد ناحیه، تقریباً 2000 ناحیه مورد نظر (RoI) را از تصویر ورودی استخراج، آن‌ها را تغییر اندازه (Warp) داده و هر ناحیه را به طور مستقل از شبکه کانولوشنی (ConvNet) عبور می‌دهد.

۱. به صورت تئوری توضیح دهید که چرا این فرآیند خاص منجر به یک گلوگاه محاسباتی (Computational Bottleneck) عظیم در زمان آموزش و تست می‌شود.
۲. معماری Fast R-CNN چگونه با تغییر ترتیب اعمال لایه‌های کانولوشنی، این محاسبات تکراری و زائد را برطرف می‌کند؟

(پ) همترازی ابعاد از طریق RoI Pooling

از آنجا که Fast R-CNN لایه‌های کانولوشنی را به یک‌باره روی کل تصویر ورودی اعمال می‌کند، باید ویژگی‌های مربوط به هر پیشنهاد ناحیه را مستقیماً از نقشه ویژگی نهایی (Final Feature Map) استخراج کند. با این حال، لایه‌های کاملاً متصل (FC) به ورودی‌هایی با اندازه ثابت نیاز دارند، در حالی که نواحی پیشنهادی می‌توانند هر اندازه دلخواهی داشته باشند.

۱. مراحل مکانیکی و عملکردی لایه «RoI Pooling» را توضیح دهید.
۲. با جزئیات شرح دهید که این لایه چگونه از مقادیر ماکزیمم در بخش‌های تعریف شده استفاده می‌کند تا پیشنهاد‌های ناحیه‌ای با اندازه‌های متغیر روی نقشه ویژگی را به یک نمایش یکنواخت و با اندازه ثابت تبدیل کند.

(ت) پارادایم شبکه واحد و Faster R-CNN

اگرچه Fast R-CNN سرعت پردازش را به شدت بهبود بخشید، اما سرعت آن در زمان تست همچنان به شدت تحت تأثیر وابستگی به الگوریتم‌های سنتی و خارجی (مانند Selective Search) برای تولید نواحی پیشنهادی بود.

۱. توصیف کنید که معماری Faster R-CNN چگونه این وابستگی خارجی را به طور کامل حذف می‌کند.

۲. وظیفه «شبکه پیشنهاد ناحیه» (RPN) را با جزئیات شرح دهید و توضیح دهید که چگونه یکپارچه‌سازی آن اجازه می‌دهد کل خط لوله تشخیص شیء به صورت سرتاسری (End-to-End) آموزش ببیند و تأخیر زمان تست را بیش از پیش کاهش دهد.

بخش عملی

(۸۰ نمره)

۱. سؤال ۱

(۴۰ نمره)

مقدمه

در این مسئله، شما با مسئله‌ی **ناحیه‌بندی تصویر** (Image Segmentation) آشنا خواهید شد و دو پارادایم متفاوت در حل این مسئله را بررسی می‌کنید. در رویکرد کلاسیک یادگیری عمیق، مدل باید روی داده‌های برچسب‌خورده آموزش ببیند تا بتواند مرز دقیق اشیاء را پیش‌بینی کند. در مقابل، مدل‌های بنیادین جدید مانند Segment Anything Model (SAM) قادرند بدون آموزش مجدد روی یک دیتاست خاص، به صورت zero-shot ماسک اشیاء را استخراج کنند.

در این تمرین، شما یک مدل U-Net را روی زیرمجموعه‌ی ناحیه‌بندی مجموعه داده Oxford-IIIT Pet آموزش خواهید داد و سپس عملکرد آن را با خروجی‌های SAM مقایسه می‌کنید. هدف این تمرین آشنایی با خط لوله‌ی کامل یک مسئله‌ی ناحیه‌بندی شامل آماده‌سازی داده‌ها، پیاده‌سازی مدل، آموزش، ارزیابی کمی و تحلیل کیفی نتایج است.

بخش ۱: ایجاد دیتاست و آماده‌سازی داده‌ها

مدل‌های ناحیه‌بندی عمیق برای آموزش به داده‌هایی با ساختار مشخص نیاز دارند. در بسیاری از مسائل عملی، داده‌ها نیازمند پیش‌پردازش و استانداردسازی هستند تا بتوان آن‌ها را به طور مؤثر در یک مدل یادگیری عمیق استفاده کرد.

در این بخش شما مجموعه داده Oxford-IIIT Pet را آماده‌سازی کرده و آن را برای استفاده در مدل U-Net فرمت می‌کنید.

وظیفه ۱: بارگذاری و بررسی داده‌ها

- **هدف شما:** مجموعه داده Oxford-IIIT Pet را دریافت و ساختار آن را بررسی کنید.
- این مجموعه داده شامل تصاویر حیوانات خانگی و ماسک‌های متناظر برای ناحیه‌بندی آن‌ها است.
- چند نمونه تصویر و ماسک متناظر را نمایش دهید تا ساختار داده‌ها مشخص شود.
- بررسی کنید که مقادیر موجود در ماسک‌ها چه معنا و ساختاری دارند.

وظیفه ۲: پیش‌پردازش داده‌ها

- **هدف شما:** داده‌ها را برای استفاده در یک مدل ناحیه‌بندی آماده کنید.
- اندازه‌ی تصاویر و ماسک‌ها را به ابعاد ثابتی مانند 256×256 تغییر دهید.
- مقادیر پیکسل‌های تصویر را نرمال‌سازی کنید.
- ماسک‌ها را به صورت **Segmentation Binary** تبدیل کنید، به طوری که تنها دو کلاس داشته باشیم:
 - 1: ناحیه حیوان
 - 0: پس‌زمینه
- داده‌ها را به سه بخش train، validation و test تقسیم کنید.

وظیفه ۳: افزایش داده (Data Augmentation)

- **هدف شما:** چند تبدیل مناسب برای افزایش تنوع داده‌ها طراحی کنید.
- تبدیل‌های پیشنهادی:
 - تقارن افقی (Horizontal Flip)
 - چرخش‌های کوچک
 - تغییرات جزئی در روشنایی یا کنتراست
- توجه کنید که هر تبدیلی که روی تصویر اعمال می‌شود باید به همان شکل روی ماسک نیز اعمال شود.

بخش ۲: پیاده‌سازی مدل U-Net

در این بخش شما معماری U-Net را برای حل مسئله‌ی ناحیه‌بندی پیاده‌سازی خواهید کرد. این معماری یکی از مدل‌های پایه و بسیار پرکاربرد در مسائل segmentation است.

وظیفه ۴: پیاده‌سازی معماری

- **هدف شما:** معماری کامل U-Net را در PyTorch پیاده‌سازی کنید.
- مدل باید شامل بخش‌های زیر باشد:
 - Encoder برای استخراج ویژگی‌ها
 - Decoder برای بازسازی نقشه ناحیه‌بندی
 - Skip Connections بین سطوح متناظر encoder و decoder
- لایه خروجی باید یک نقشه احتمال برای کلاس شیء تولید کند.

وظیفه ۵: تابع هزینه و معیارهای ارزیابی

- هدف شما: یک تابع هزینه مناسب برای آموزش مدل انتخاب کنید.
- می‌توانید از یکی از گزینه‌های زیر استفاده کنید:
 - Binary Cross Entropy
 - Dice Loss
 - ترکیبی از این دو
- معیارهای زیر را نیز برای ارزیابی پیاده‌سازی کنید:
 - Intersection over Union (IoU)
 - Dice Score

وظیفه ۶: آموزش مدل

- هدف شما: حلقه آموزش مدل را پیاده‌سازی کنید.
- در هر تکرار:
 - یک دسته از تصاویر و ماسک‌ها را به مدل بدهید
 - مقدار loss را محاسبه کنید
 - گرادیان‌ها را با استفاده از backpropagation به‌روزرسانی کنید
- آموزش را برای چندین epoch اجرا کرده و مقدار training loss و validation loss را ثبت کنید.

وظیفه ۷: تحلیل روند آموزش

- نمودارهای زیر را رسم کنید:
 - Training Loss
 - Validation Loss
 - Validation IoU
- تحلیل کوتاهی درباره روند همگرایی مدل ارائه دهید.

بخش ۳: ارزیابی کیفی و کمی مدل U-Net

وظیفه ۸: ارزیابی کمی

- مدل آموزش‌دیده را روی مجموعه test اجرا کنید.
- معیارهای زیر را گزارش دهید:
 - میانگین IoU
 - میانگین Dice Score

وظیفه ۹: ارزیابی کیفی

- برای چند تصویر از مجموعه آزمون، موارد زیر را کنار هم نمایش دهید:
 - تصویر اصلی
 - ماسک واقعی
 - ماسک پیش‌بینی شده توسط مدل
- برای درک بهتر نتایج، ماسک پیش‌بینی شده را به صورت overlay روی تصویر اصلی نیز نمایش دهید.

وظیفه ۱۰: تحلیل خطا

- چند نمونه‌ای که مدل در آن‌ها خطای قابل توجهی دارد انتخاب کنید.
- بررسی کنید که علت احتمالی خطا چیست. برای مثال:
 - پس‌زمینه پیچیده
 - شباهت رنگ شیء و پس‌زمینه
 - مرزهای مبهم

بخش ۴: ناحیه‌بندی Zero-Shot با SAM

در این بخش شما از مدل Segment Anything Model (SAM) استفاده می‌کنید تا بدون آموزش مجدد، ناحیه‌بندی اشیاء را انجام دهید.

وظیفه ۱۱: بارگذاری مدل

- **هدف شما:** مدل SAM را بارگذاری کرده و برای انجام استنتاج آماده کنید. در این تمرین از حالت Automatic Mask Generation استفاده می‌کنیم که در آن مدل بدون نیاز به هیچ prompt از طرف کاربر، مجموعه‌ای از ماسک‌های پیشنهادی برای نواحی مختلف تصویر تولید می‌کند.

وظیفه ۱۲: تولید ماسک‌ها با Automatic Mask Generation

- از کلاس SamAutomaticMaskGenerator استفاده کنید تا برای هر تصویر مجموعه‌ای از ماسک‌های کاندید تولید شود.
- توجه داشته باشید که مدل ممکن است تعداد زیادی ماسک برای بخش‌های مختلف تصویر (اشیاء، اجزای اشیاء یا حتی بخش‌هایی از پس‌زمینه) تولید کند.
- تمام ماسک‌های تولید شده را روی تصویر نمایش دهید تا رفتار مدل در قطعه‌بندی تصویر مشاهده شود.

وظیفه ۱۳: انتخاب و ارزیابی بهترین ماسک

- از میان ماسک‌های تولید شده، ماسکی را انتخاب کنید که بیشترین تطابق را با ماسک واقعی (Ground Truth) دارد.
- برای این منظور مقدار Intersection over Union (IoU) را برای هر ماسک محاسبه کنید و ماسکی را که بیشترین مقدار IoU دارد به عنوان خروجی نهایی در نظر بگیرید.
- سپس خروجی انتخاب‌شده را با ماسک واقعی مقایسه کرده و معیارهای زیر را محاسبه کنید:
 - IoU
 - Dice Score

بخش ۵: طراحی آزمایش و مقایسه مدل‌ها

در این بخش شما عملکرد مدل آموزش‌دیده U-Net را با مدل SAM مقایسه خواهید کرد.

آزمایش الف: مقایسه کیفی

- برای چند تصویر از مجموعه آزمون، نتایج زیر را کنار هم نمایش دهید:
 - تصویر اصلی
 - ماسک واقعی
 - خروجی U-Net
 - خروجی SAM

آزمایش ب: مقایسه کمی

- میانگین IoU و Dice هر دو مدل را محاسبه کنید.
- نتایج را در قالب یک جدول مقایسه‌ای گزارش دهید.

آزمایش ج: تحلیل نتایج

- در یک پاراگراف تحلیلی به سوالات زیر پاسخ دهید:
 - در چه شرایطی مدل U-Net عملکرد بهتری دارد؟
 - در چه شرایطی SAM عملکرد قابل قبولی بدون آموزش ارائه می‌دهد؟
 - مزایا و محدودیت‌های هر رویکرد چیست؟

جمع‌بندی

در پایان این تمرین باید بتوانید یک خط لوله کامل برای مسئله‌ی ناحیه‌بندی تصویر طراحی کنید، یک مدل عمیق را آموزش دهید، از یک مدل بنیادین به صورت zero-shot استفاده کنید و نتایج حاصل را به صورت کمی و کیفی تحلیل نمایید.

مقدمه

در این مسئله، شما به سمت جریان‌های کاری آماده برای محیط عملیاتی (production-ready) حرکت خواهید کرد. شما دو تشخیص‌دهنده یک‌مرحله‌ای سبک و پیشرفته، یعنی SSDLite (Torchvision) و YOLOv8n (Ultralytics) را بر روی زیرمجموعه‌ای از مجموعه داده PASCAL VOC 2007 تنظیم دقیق خواهید کرد. در پایان این مسئله، یاد می‌گیرید که چگونه داده‌ها را برای چارچوب‌های (frameworks) مختلف فرمت کنید، معماری‌های از پیش آموزش‌دیده را برای دامنه‌های سفارشی تغییر دهید، و آزمایش‌های تجربی برای ارزیابی مبادلات (trade-offs) مدل‌ها طراحی کنید.

بخش ۱: ایجاد دیتاست و آماده‌سازی داده‌ها

تشخیص‌دهنده‌های مدرن اغلب به خطوط لوله داده با فرمت‌بندی دقیق نیاز دارند. در دنیای واقعی، مهندسان بینایی ماشین به ندرت از دیتاست‌های خام به همان شکل اولیه استفاده می‌کنند؛ بلکه داده‌ها را بر اساس نیاز مسئله فیلتر و پالایش می‌کنند. در این بخش، شما دیتاست سفارشی خود را از روی مجموعه داده PASCAL VOC 2007 ایجاد خواهید کرد.

برای تسریع آموزش و تمرکز بر روی یک دامنه خاص، ما یک زیرمجموعه تحت عنوان «فقط وسایل نقلیه» (Vehicles Only) شامل چهار کلاس ایجاد می‌کنیم: ماشین (Car)، اتوبوس (Bus)، دوچرخه (Bicycle)، و موتورسیکلت (Motorbike).

وظیفه ۱: فیلتر کردن و استخراج داده‌ها (ایجاد دیتاست سفارشی)

مجموعه داده PASCAL VOC 2007 دارای ۲۰ کلاس است. اولین قدم شما، نوشتن اسکریپتی است که کل این مجموعه داده را بررسی کرده و تصاویری را که حداقل یکی از ۴ کلاس هدف ما را دارند، جدا کند.

- **تقسیم‌بندی داده‌ها (Data Splits):** شما باید ساختار استاندارد مجموعه داده را حفظ کنید. تصاویری که از پوشه Train استخراج می‌کنید را برای آموزش مدل، و تصاویری که از پوشه Val استخراج می‌کنید را برای اعتبارسنجی و ارزیابی (Evaluation) استفاده کنید.
- **نکته‌ای در مورد عدم توازن کلاس‌ها (Class Imbalance):** در مسائل واقعی تشخیص شیء، توزیع کلاس‌ها به ندرت متوازن است. به عنوان مثال، در این مجموعه داده، تعداد تصاویر شامل ماشین (حدود ۷۱۳ تصویر در Train/Val) بسیار بیشتر از تصاویر شامل اتوبوس (حدود ۱۸۶ تصویر) است. همچنین یک تصویر ممکن است همزمان شامل چندین شیء از کلاس‌های مختلف باشد. **نیازی به حذف تصاویر برای بالانس کردن دقیق کلاس‌ها نیست.**

راهنمای آماری: پس از فیلتر کردن مجموعه *Train/Val*، انتظار می‌رود توزیع اشیاء شما تقریباً به شکل زیر باشد (از این آمار برای بررسی صحت اسکریپت خود استفاده کنید):

• Car: ۱۲۵۰ نمونه (Instances)

• Bus: ۲۲۹ نمونه

• Bicycle: ۳۵۳ نمونه

• Motorbike: ۳۳۹ نمونه

وظیفه ۲: فرمت PyTorch/Torchvision

مدل‌های تشخیص Torchvision انتظار دارند اهداف (targets) به شکل لیستی از دیکشنری‌ها ساختاریافته باشند.

• **هدف شما:** یک پوشش (wrapper) سفارشی از نوع Dataset در PyTorch بنویسید که روی دیتاست فیلترشده شما (خروجی وظیفه ۱) کار کند و حاشیه‌نویسی‌های XML را تجزیه (parse) نماید.

• **الزام فرمت:** برای هر تصویر، جعبه‌های محدودکننده را در فرمت $(x_{tl}, y_{tl}, x_{br}, y_{br})$ استخراج کرده و برچسب‌های رشته‌ای را به اعداد صحیح (مثلاً ۰ تا ۳ برای ۴ کلاس وسایل نقلیه) نگاشت کنید. یک دیکشنری شامل کلیدهای "boxes" (یک FloatTensor) و "labels" (یک Int64Tensor) برگردانید.

وظیفه ۳: فرمت Ultralytics/YOLO

کتابخانه Ultralytics YOLO از یک ساختار حاشیه‌نویسی کاملاً متفاوت و مبتنی بر متن استفاده می‌کند.

• **هدف شما:** اسکریپتی بنویسید که دیتاست فیلترشده شما را به فرمت پوشه‌بندی و فایل‌های txt مخصوص YOLO تبدیل کند (جدا کردن images/train و images/val و به همین ترتیب برای labels).

• **الزام فرمت:** هر تصویر باید یک فایل txt. متناظر داشته باشد که در آن هر خط نمایانگر یک شیء با این فرمت است: [class_id x_center y_center width height]. توجه داشته باشید که این مختصات جعبه محدودکننده باید نسبت به ابعاد تصویر بین ۰ و ۱ نرمال‌سازی شوند. در نهایت، فایل پیکربندی data.yaml مورد نیاز را برای تعریف مسیرهای دیتاست خود و نام ۴ کلاس هدف تولید کنید.

بخش ۲: بررسی دقیق‌تر - تصویرسازی لنگرها (Priors)

پیش از آنکه با این مدل‌ها کاملاً به عنوان یک جعبه سیاه (black box) برخورد کنیم، فرضیات جغرافیایی داخلی آن‌ها را بررسی می‌کنیم. SSDLite، مانند بسیاری از تشخیص‌دهنده‌های یک‌مرحله‌ای، تبدیل‌هایی را پیش‌بینی می‌کند که جعبه‌های لنگر از پیش تعریف شده (یا priors) را به ناحیه‌های پیشنهادی (region proposals) تبدیل می‌کند.

وظیفه ۳: استخراج لنگرها (Anchor Extraction)

• **هدف شما:** مدل ssdlite320_mobilenet_v3_large را از torchvision نمونه‌سازی کنید. یک تنسور ساختگی (نشان‌دهنده یک تصویر با ابعاد 3x320x320) را به anchor_generator

مدل پاس دهید تا جعبه‌های prior خام تولید شده برای نقشه‌های ویژگی (feature maps) را استخراج کنید.

وظیفه ۴: تصویرسازی (Visualization)

• **هدف شما:** ما باید موقعیت مکانی مرکز هر سلول در شبکه نقشه ویژگی را بدانیم. جعبه‌های لنگر استخراج شده متناظر با **مرکزی‌ترین سلول شبکه** را روی یک تصویر نمونه از PASCAL VOC بیندازید تا چندین جعبه با نسبت‌های ابعاد (aspect ratios) مختلف را مشاهده کنید که در مرکز تصویر با هم تلاقی دارند.

بخش ۳: تنظیم ظریف SSDLite (Torchvision)

اکنون شما یک مدل از پیش آموزش‌دیده SSDLite با معماری پایه MobileNetV3-Large را برای کار خاص تشخیص وسایل نقلیه ما سازگار خواهید کرد.

وظیفه ۵: تغییر معماری (Architecture Modification)

• **هدف شما:** مدل از پیش آموزش‌دیده SSDLite را بارگذاری کنید. از آنجا که مدل اصلی روی مجموعه داده COCO (۸۰ کلاس) آموزش دیده است، شما باید سر طبقه‌بندی (head.classification_head.num_classes) را طوری تغییر دهید که با زیرمجموعه خاص ما (۴ کلاس وسیله نقلیه + ۱ کلاس پس‌زمینه) مطابقت داشته باشد. وزن‌های ستون فقرات (backbone) را منجمد (freeze) کنید تا مطمئن شوید که ما فقط سر تشخیص (detection head) را آموزش می‌دهیم.

وظیفه ۶: مسیر پیش‌رو (The Forward Pass)

• **هدف شما:** حلقه اصلی آموزش را کامل کنید. یک دسته (batch) از تصاویر و دیکشنری‌های هدف فرمت‌شده خود را به مدل پاس دهید. SSDLite به طور خودکار زیان‌های طبقه‌بندی و رگرسیون جعبه محدودکننده را محاسبه خواهد کرد. این زیان‌های کلی را استخراج کرده و برای به‌روزرسانی وزن‌های مدل، متد backward() را فراخوانی کنید.

بخش ۴: تنظیم ظریف YOLOv8n (Ultralytics)

در ادامه، شما از یک رابط برنامه‌نویسی (API) سطح بالا و مدرن برای آموزش YOLOv8 Nano استفاده خواهید کرد. توجه کنید که چگونه کتابخانه Ultralytics حلقه آموزش PyTorch را که در بخش ۳ پیاده‌سازی کردید، از دید شما پنهان می‌کند.

وظیفه ۷: خط لوله آموزش (Training Pipeline)

• **هدف شما:** اسکریپت پایتون را با استفاده از کلاس YOLO بنویسید تا وزن‌های از پیش آموزش‌دیده yolo8n.pt را بارگذاری کنید. فرآیند آموزش را با استفاده از فایل data.yaml که در بخش ۱ تولید کردید، آغاز کنید. مطمئن شوید که اندازه تصویر را روی ۳۲۰ تنظیم کرده‌اید (برای مقایسه عادلانه با SSDLite) و آموزش را برای ۵۰ دوره (epochs) انجام دهید.

بخش ۵: طراحی آزمایش و ارزیابی

در این بخش پایانی، شما آزمایش‌هایی را برای ارزیابی دقیق و مقایسه رفتار مدل‌های آموزش‌دیده خود طراحی خواهید کرد.

آزمایش الف: آزمون بیش‌برازش (The Overfit Test) (وظیفه ۸)

قبل از تعهد به یک اجرای طولانی مدت آموزش، رویه خوبی است که اطمینان حاصل کنید مدل و خط لوله داده شما قادر به یادگیری هستند.

• **هدف شما:** یک زیرمجموعه بسیار کوچک از داده‌های خود دقیقاً شامل ۱۰ تصویر ایجاد کنید. هم SSDLite و هم YOLOv8n را روی این مجموعه داده خرد برای ۵۰ دوره آموزش دهید. نمودارهای زیان آموزش را رسم کرده و پیش‌بینی‌های نهایی را روی همین ۱۰ تصویر تصویرسازی کنید. شما باید به طور تجربی ثابت کنید که مدل‌ها مکان‌های فضایی و کلاس‌های این مجموعه داده کوچک را با موفقیت حفظ (memorize) می‌کنند.

آزمایش ب: تنظیم سرکوب غیربیشینه (NMS Tuning) (وظیفه ۹)

تشخیص‌دهنده‌های یک مرحله‌ای به طور همزمان چندین جعبه محدودکننده هم‌پوشان را پیش‌بینی می‌کنند و برای فیلتر کردن تشخیص‌های اضافی به NMS نیاز دارند.

• **هدف شما:** یک تصویر بسیار شلوغ از مجموعه اعتبارسنجی (validation set) انتخاب کنید. استنتاج (inference) را با استفاده از مدل YOLOv8n آموزش‌دیده خود در سه آستانه مختلف IoU در NMS اجرا کنید: 0.1، 0.5، و 0.9. سه تصویر حاصل را در کنار هم رسم کنید و تحلیل مختصری بنویسید که تفاوت‌های بصری در رفتار سرکوب را توضیح دهد.

آزمایش ج: سرعت استنتاج در برابر دقت (وظیفه ۱۰)

مهندسی بینایی ماشین در دنیای واقعی، یک عمل متعادل‌سازی بین هزینه محاسباتی و دقت تشخیص است.

• **هدف شما:** مجموعه اعتبارسنجی خود را با استفاده از هر دو مدل تنظیم ظریف شده SSDLite و YOLOv8n پیمایش کنید. میانگین سرعت استنتاج را بر حسب فریم بر ثانیه (FPS) محاسبه کرده و میانگین دقت متوسط نهایی (mAP) را برای هر دو مدل ارزیابی کنید.

- **خروجی قابل ارائه:** این معیارها را در یک جدول مقایسه‌ای جمع‌آوری کنید. یک پاراگراف نتیجه‌گیری بنویسید که این مبادلات را تحلیل کند: چرا ممکن است یک مدل فریم‌ها را سریع‌تر پردازش کند و این موضوع چگونه بر دقت جعبه‌های محدودکننده آن تأثیر می‌گذارد؟

VGG16 - Structural Details												
#	Input Image			output			Layer	Stride	Kernel			Param
1	224	224	3	224	224	64	conv3-64	1	3	3	3	1792
2	224	224	64	224	224	64	conv3064	1	3	3	64	36928
	224	224	64	112	112	64	maxpool	2	2	2	64	0
3	112	112	64	112	112	128	conv3-128	1	3	3	64	73856
4	112	112	128	112	112	128	conv3-128	1	3	3	128	147584
	112	112	128	56	56	128	maxpool	2	2	2	128	65664
5	56	56	128	56	56	256	conv3-256	1	3	3	128	295168
6	56	56	256	56	56	256	conv3-256	1	3	3	256	590080
7	56	56	256	56	56	256	conv3-256	1	3	3	256	590080
	56	56	256	28	28	256	maxpool	2	2	2	256	0
8	28	28	256	28	28	512	conv3-512	1	3	3	256	1180160
9	28	28	512	28	28	512	conv3-512	1	3	3	512	2359808
10	28	28	512	28	28	512	conv3-512	1	3	3	512	2359808
	28	28	512	14	14	512	maxpool	2	2	2	512	0
11	14	14	512	14	14	512	conv3-512	1	3	3	512	2359808
12	14	14	512	14	14	512	conv3-512	1	3	3	512	2359808
13	14	14	512	14	14	512	conv3-512	1	3	3	512	2359808
	14	14	512	7	7	512	maxpool	2	2	2	512	0
14	1	1	25088	1	1	4096	fc		1	1	25088	102764544
15	1	1	4096	1	1	4096	fc		1	1	4096	16781312
16	1	1	4096	1	1	1000	fc		1	1	4096	4097000
Total												138,423,208

شکل ۱: جزئیات ساختاری شبکه VGG16